МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №4

«Контекст, стан і кеш агента»

з дисципліни: «Обчислювальні методи»

Варіант 11 (Предметна область: «Бібліотека»)

Виконав: студент групи AI-243

Іпатов Богдан Сергійович

Варіант: **11 (Бібліотека)**

Перевірив: ас. Баранов О.М.

Одеса – 2026

1. Мета роботи, номер варіанта, предметна область, модель і розмір вікна

Мета роботи: дослідити поведінку та вартість виконання автономного агента на довгій задачі з багаторазовими зверненнями до інструментів; визначити практичні межі контекстного вікна мовної моделі та виявити наслідки його переповнення; реалізувати й експериментально порівняти механізми захисту від переповнення контексту (`trim`, `trim+state`, `summary`) і кешування звернень (`cache`); проаналізувати динаміку споживання токенів і структуру викликів у системі трасування `Langfuse`.

Параметр конфігурації	Значення
Варіант	11
Предметна область	Бібліотека (каталог книг, умови видачі, розрахунок штрафів)
Базова модель	`qwen2.5:3b` у середовищі Ollama (`TEMPERATURE = 0`)
Початкове вікно моделі	32 768 токенів (вихідний контекст архітектури Qwen 2.5)
Зменшене вікно (крок 1)	4 096 токенів (`qwen25-4k`, файл `Modelfile`)
Робоче експериментальне вікно	3 072 токени (`qwen25-3k`, файл `Modelfile_3k`, випадок C)
Поріг початку стиснення	`COMPRESS_AT = 1100` токенів
Глибина збереження хвоста	`KEEP_LAST = 6` повідомлень (`STATE_RESULT_CHARS = 200` символів)
Журналювання	Загальний модуль `utils/lab_logger.py` (файли в `logs/`)

Обґрунтування вибору COMPRESS_AT = 1100 (`config.py`)

У базовій методиці наводиться орієнтовний поріг `COMPRESS_AT = 2150` (~70% від вікна `3072`). Проте в нашій предметній області модель `qwen25-3k` вже на першому кроці (`prompt_tokens = 1206`) пакетно генерує одразу 12 паралельних викликів інструментів (`6 × search_books` і `6 × book_info`), через що на 2-му кроці контекст зростає до `1 947–1 961` токенів і прогін завершується ще до порогу `2 150`. Щоб стиснення гарантовано активувалося одразу після першого кроку (`1206 > 1100`) і дозволило порівняти всі три стратегії керування історією, встановлено `COMPRESS_AT = 1100`:

```
# COMPRESS_AT = 1100 (нижче 2150 з методички): у qwen25-3k на 1-му кроці prompt_tokens = 1206
# і модель пакетно викликає одразу 12 інструментів, завершуючи задачу на 2-му кроці (1961 < 2150).
# Поріг 1100 гарантує спрацювання стиснення одразу після 1-го кроку (1206 > 1100) для порівняння режимів.
COMPRESS_AT = 1100      # скорочуємо, коли prompt_tokens перевищив це число
KEEP_LAST = 6           # скільки останніх повідомлень лишати при обрізанні
```

2. Довга задача (`make_task.py`)

Скриптом `make_task.py` згенеровано довгу задачу для варіанта 11 (`long_task.json`) за шістьма книгами бібліотечного каталогу: «Основи алгоритмів» (7 дн., 10 грн/день), «Зоряний експрес» (14 дн., 5 грн/день), «Місто вічної ночі» (14 дн., 8 грн/день), «Кулінарія для лінивих» (14 дн., 5 грн/день), «Хроніки Марсіанської бази» (14 дн., 8 грн/день) та «Чистий код у Python» (7 дн., 15 грн/день).

> Формулювання завдання: «Для кожної з шести книжок: Основи алгоритмів, Зоряний експрес, Місто вічної ночі, Кулінарія для лінивих, Хроніки Марсіанської бази, Чистий код у Python. Для кожної: (а) знайди в корпусі, про що книжка, і наведи одним реченням; (б) знайди в довіднику термін видачі та штраф за день; (в) порахуй розмір штрафу за 3 дні затримки. У кінці назви загальну суму штрафу для усіх шести книжок одним числом.»

Еталонний аналітичний розрахунок: `3×10 + 3×5 + 3×8 + 3×5 + 3×8 + 3×15 = 30 + 15 + 24 + 15 + 24 + 45 = 153 грн` (`expected = "153"`, теоретично очікуваний послідовний мінімум інструментів: `min_calls = 18` — 6 викликів `search_books` + 6 `book_info` + 6 `calculate`).

3. Таблиця 2, показники та аналіз поведінки моделі (`probe_none_r1.log`)

Після запуску `python measure.py none 1 probe` лог `logs/probe_none_r1.log` оброблено скриптом `analyze.py`. Покроковий рух вхідних токенів наведено в Таблиці 2.

Таблиця 2 – Зростання входу в прогоні без захисту (`probe_none_r1`)

Крок	`prompt_tokens`	Приріст	Що зробила модель
1	1 206	—	Згенерувала одразу 12 паралельних викликів: 6 `search_books` та 6 `book_info` для всіх шести книг
2	1 954	+748	Отримавши 12 відповідей від інструментів, дала фінальну текстову відповідь без виклику `calculate`

Аналітичні показники `analyze.py` (крок 4, пункти а–д):

- **(а) Вхід першого кроку (`C`):** `1 206` токенів (системний промпт, схеми 4 інструментів і довга задача).
- **(б) Типовий приріст за крок (`d`):** `748` токенів у `probe_none_r1` (відповіді 12 викликів).

- **(в) Сума `prompt_tokens` та формула:** фактична сума `1 206 + 1 954 = 3 160` токенів; за квадратичною формулою `n*C + d*n*(n-1)/2 = 2*1206 + 748*2*1/2 = 3 160` токенів — **збіг точний (похибка 0%)**.
- **(г) Крок зупинки росту:** `не помічено` — максимальний вхід сягнув `1 954` токенів (64% вікна `3 072`), і прогін завершився статусом `final` на 2-му кроці.
- **(д) Поведінка моделі та класифікація (Випадок С):** початково у вікні `NUM_CTX = 4096` (`qwen25-4k`) вхід складав < 50% вікна, тому згідно з методикою (**Випадок С: «Задача закоротка»**) вікно звужено через `Modelfile_3k` до `3 072` (`qwen25-3k`). Без виклику `calculate` модель схильна до арифметичних галюцинацій (у `main_none_r2` назвала `158 грн`, у `main_none_r3` — `-2.0`).

4. Обрізання історії та стан задачі (`probe_trim`, `probe_trim+state`)

Таблиця 3 – Стиснення історії (`probe_*_r1.log`)

Конфігурація	Кроки стиснення	Вхід до стиснення	Повідомлень було -> стало	Повторних викликів	Статус / Кроків
`probe_none_r1`	—	—	—	0	`final` / 2
`probe_trim_r1`	1..25 (щокроку)	1 206	15 -> 2	**288**	`step_limit` / 25
`probe_trim+state_r1`	1, 2, 3, 4	1 206 -> 1 750 -> 2 255 -> 2 547	15 -> 2 (стан), 16 -> 2, 6 -> 6, 8 -> 8	**11**	`final` / 5

Чому `15 -> 2` *замість* `6` *останніх повідомлень:* після пакету з 12 викликів масив `rest = messages[2:]` містить 1 `assistant` із 12 `tool_calls` і 12 відповідей `tool`. Щоб не порушити протокол ролей OpenAI, цикл `while cut < len(rest) and rest[cut]["role"] == "tool"` у `compress.py` просуває зріз до кінця блоку, відкидаючи весь пакет (`15 -> 2`). Без збереження стану модель на кроках 2..25 бачила чистий запит і повторювала ті самі 12 викликів **25 разів (288 повторів)**.

Аналіз блоку стану (`probe_trim+state_r1.log`, п. 7 кроку 6):

```
2026-11-14 00:05:57 - INFO -   СТИСНЕННЯ: вхід був 1206 токенів, повідомлень 15 -> 2, стан додано
Результати вже виконаних викликів (збережені програмою):
- search_books({"query": "Основи алгоритмів"}) -> Основи алгоритмів
- book_info({"title": "Основи алгоритмів"}) -> Примірників: 2, термін видачі: 7 днів, штраф: 10 грн/день
... (разом 12 рядків: 6 пошуків + 6 довідок) ...
- book_info({"title": "Чистий код у Python"}) -> Примірників: 4, термін видачі: 7 днів, штраф: 15 грн/день
```

Блок стану після 1-го кроку містить рівно **12 рядків** загальною довжиною **1 119 символів** (~**448 токенів** за правилом 2,5 символа/токен; разом з умовою задачі — 1 548 символів). Реальний вхід 2-го кроку в `probe_trim+state_r1` склав `1 750` токенів (+544 до `1 206`), тоді як сирі 12 повідомлень у `probe_none_r1` дали +748 токенів (`1 954 - 1 206`). Тобто `TaskState` заощадив **~204 токени (~27%)**, скоротив кількість кроків з **25 до 5**, а повторні виклики — **з 288 до 11**.

5. Кешування викликів моделі (cache1 vs cache2)

Перевірку cache.py виконано на порожній директорії cache/ двома послідовними запусками measure.py none 1 cache1 і measure.py none 1 cache2 (Таблиця 4).

Таблиця 4 – Робота кешу

Запуск	Кроків	Влучань у кеш	Викликів до Ollama	Час, с	Токенів входу	Правильно
1 (cache1_none_r1, порожній кеш)	2	0	2	**8.8**	3 153	так (153)
2 (cache2_none_r1, той самий запит)	2	**2**	0	**0.7**	3 153	так (153)

Висновки: другий запуск виконався у **12,6 раза швидше** (0.7 с проти 8.8 с), отримавши відповіді з JSON-файлів (| з кешу) без жодного звернення до Ollama. Якби перед другим запуском змінився хоча б 1 символ у SYSTEM, лавинний ефект sha256 змінив би ключ make_key, влучань у кеш було б 0, і запити пішли б на сервер заново.

6. Основне вимірювання, додаткова серія (extra) та траси Langfuse

Повне вимірювання виконано командами python measure.py all 3 main (12 унікальних логів) та python measure.py summary 3 extra (3 логи logs/extra_summary_r1..3.log). Медіани наведено в Таблиці 5 і Таблиці 6.

Таблиця 5 – Вартість циклу в основних конфігураціях (main, медіани по 3 повторах)

Конфігурація	Правильно	Кроків	Вхід, ток.	Вихід, ток.	Підсумов., ток.	Час, с	Стиснень	Повторів
none	1 з 3	2	3 160	834	0	5.9	0	0
trim	0 з 3	25	30 150	7 875	0	53.1	25	288
trim+state	0 з 3	6	12 681	2 066	0	14.3	4	12
summary (main)	0 з 3	8	15 420	2 599	6 075	32.1	7	16

Таблиця 6 – Результати додаткового завдання (extra_summary_r1..3)

Конфігурація	Правильно	Кроків	Вхід, ток.	Вихід, ток.	Підсумов., ток.	Час, с	Стиснень	Повторів
summary (медіана з 3)	0 з 3	6	12 089	1 791	2 895	22.1	4	7

Аналіз знайдених поламок у 15 прогонах (`main` + `extra_summary`):

- **Зациклення при сліпому обрізанні (`step_limit`):** усі **3** з **3** прогонів `main_trim` вперлися в ліміт 25 кроків (`288` повторних викликів, `30 150` вхідних токенів) без фінальної відповіді.
- **Дегенерація генерації до ліміту (`retry_limit`):** `main_trim+state_r1` (92 927 вихідних токенів, 625.2 с) та `extra_summary_r2` (92 891 вихідний токен, 631.7 с) увійшли в циклічну генерацію всередині одного кроку.
- **Передчасне завершення (`< 19` кроків) та числові помилки:** усі 12 решти прогонів завершилися за `2..9` кроків; правильну суму **`153`** дав лише **`1 з 15`** (`main_none_r1`). В інших: `main_none_r2` вивів `158`, `main_none_r3` — `-2.0`, `main_trim+state_r2` — `27.0`, `main_summary_r1` — `55`, `extra_summary_r1` — `114`.

Аналіз трас у Langfuse:

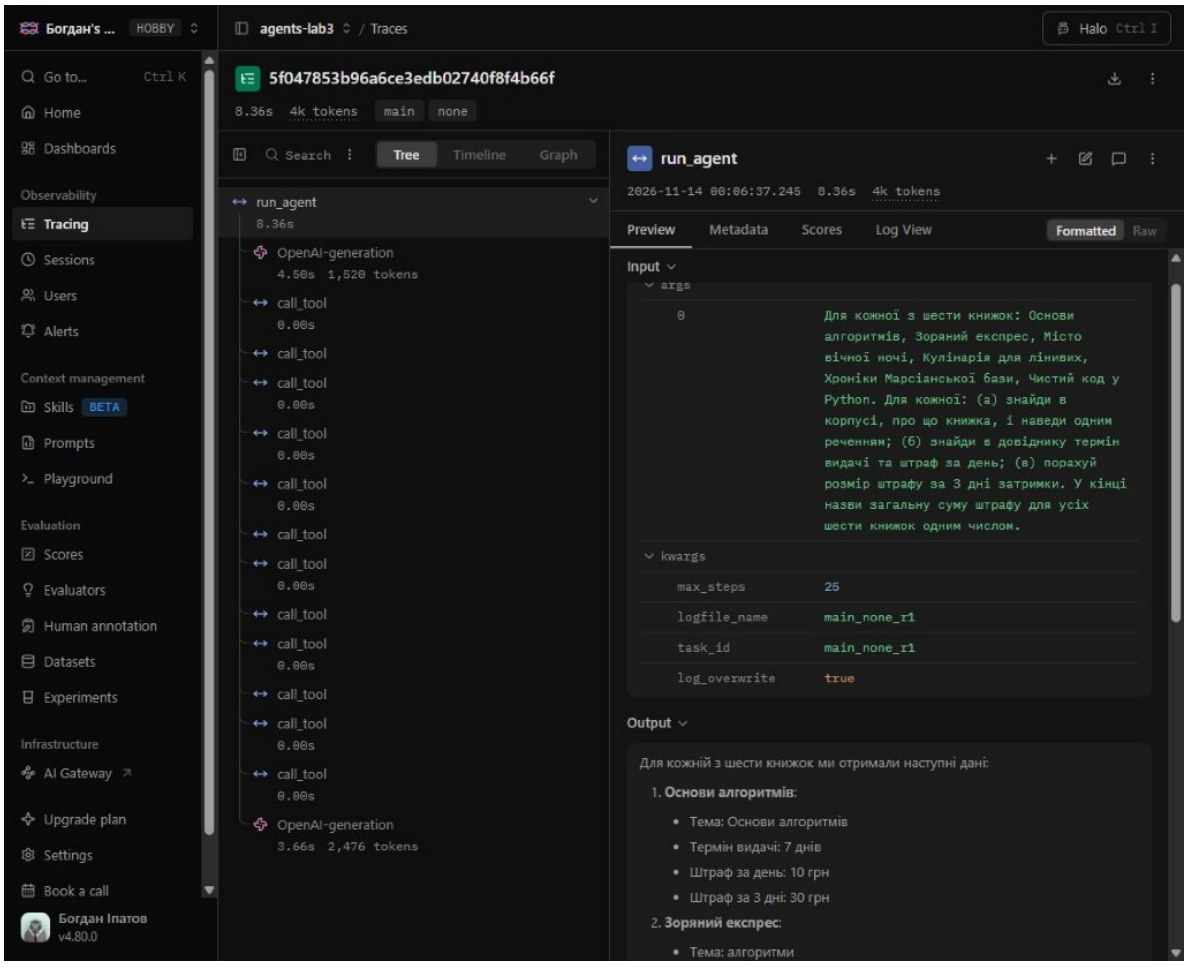


Рисунок 1 – Дерево траси `long-main_none_r1` *у веб-інтерфейсі Langfuse*

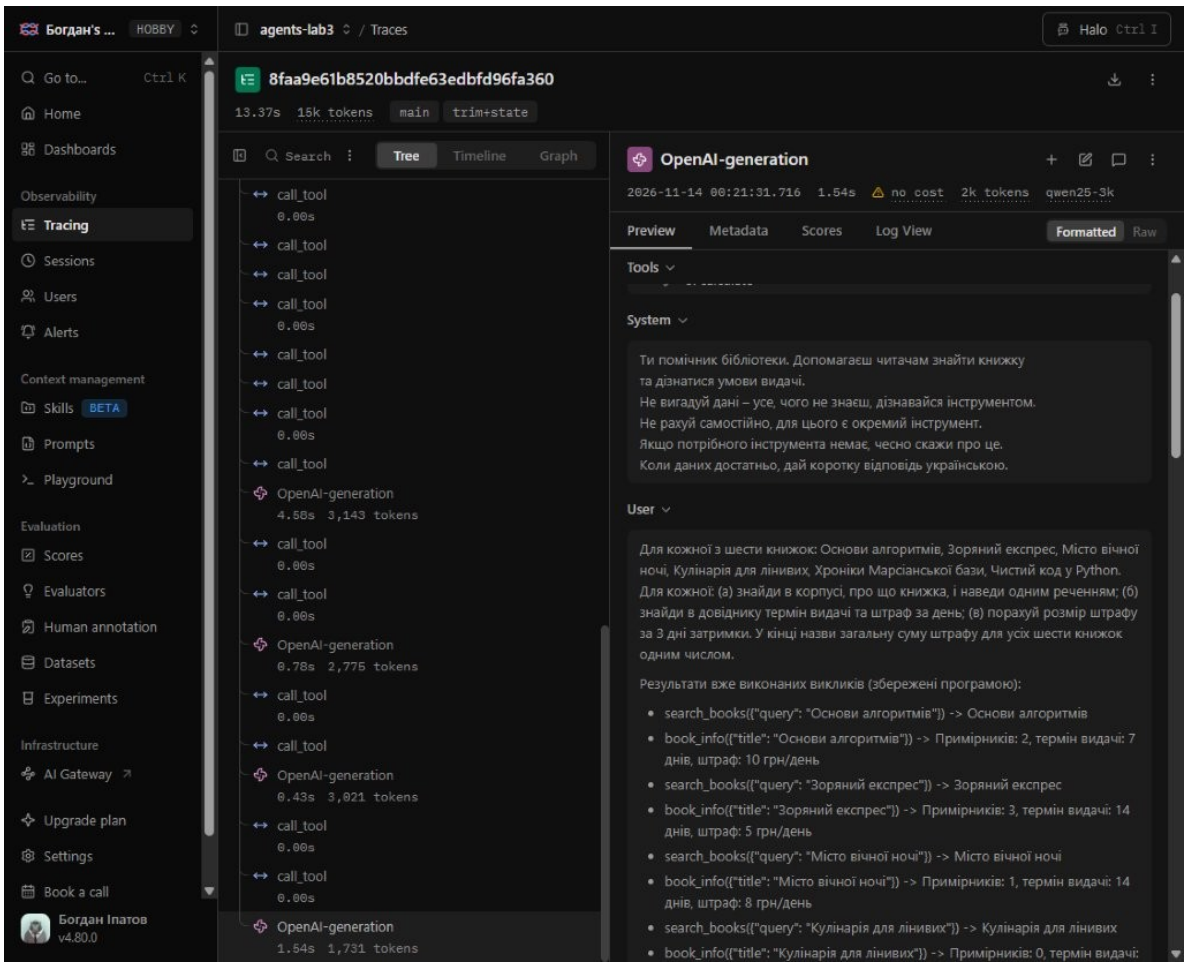


Рисунок 2 – Дерево траси `long-main_trim+state` *у веб-інтерфейсі Langfuse*

У трасі `main_trim+state_r3` (trace ID `8faa9e61b8520bbdfe63edbfd96fa360`) зафіксовано чіткий момент падіння вхідного контексту після стиснення: кроки 1–5 зростали `1 206 -> 1 762 -> 2 479 -> 2 725 -> 2 971` (97% вікна `3 072`), далі спрацювало стиснення (`11 -> 8` повідомлень із блоком стану), і на **кроці 6 `prompt_tokens` впав до `1 538` (-1 433 токени)**.

7. Додаткове завдання: підсумовування замість обрізання (`summary.py`)

У `summary.py` функція `compress_by_summary` виділяє старі повідомлення `old` до хвоста `KEEP_LAST = 6` і просить `qwen25-3k` (`temperature=0`) стисло переказати їх зі збереженням усіх чисел і назв. У `extra_summary_r1..3` це дало медіанну доплату **2 895** токенів (`6 075` у `main_summary`) та виявило типові спотворення малої 3B-моделі:

```
--- logs/extra_summary_r1.log (крок 1 -> 2 -> 7) ---
СТИСНЕННЯ: вхід був 1206 токенів, повідомлень 15 -> 2, підсумовування коштувало 1176 токенів
ПІДСУМОК ІСТОРІЇ:
Вище представлені результати виконання роботи з використанням інструментів для пошуку книг...
Всі числа, назви й результати інструментів були збережені без змін. [жодної ставки не названо!]
...
ВІДПОВІДЬ (крок 7): 1. Основи алгоритмів - 3.5 днів, штраф за день: 3.5 грн.
Загальний штраф за 3 дні прострочення для всіх книг: 10.5 + 10.5 + 24 + 0 + 24 + 45 = 114 грн.
```

Замість переліку ставок сумаризатор іноді видавав шаблонний переказ, а випадкове число `3.5` з кроку 2 `calculate(1 + 2.5)` під час наступного переказу втратило контекст і перетворилося на ставку штрафу. Детермінований `TaskState` (`trim+state`) на моделі 3B перевершив `summary` і за точністю фактів, і за вартістю токенів.

8. Відповіді на питання

1. На якому кроці агент без захисту дійшов до межі вікна і як ви це побачили (таблиця 2, пункти (г) і (д) кроку 4)? Що в його поведінці змінилося після цього?

У наших прогонах при `NUM_CTX = 3072` агент без захисту не перетнув 100% межу вікна через пакетний виклик усіх 12 інструментів на 1-му кроці: у `probe_none_r1` максимум становив `1 954` токени (64%), у `main_none_r3` — `2 220` токенів (72%), тому `analyze.py` показав `вхід перестав рости на кроці: не помічено` (Випадок C). Проте вже на обсязі ~2 тис. токенів `qwen2.5:3b` пропускала виклик `calculate` і вигадувала суми (`158` у `main_none_r2`, `-2.0` у `main_none_r3`). За послідовних кроків межа `3 072` була б перевищена на 4-му кроці (`1206 + 3×741 = 3429`), після чого Ollama мовчки обрізала б початок історії (`prompt_tokens` застигає), і агент втратив би інструкцію та почав циклічно повторювати дії.

2. Порівняйте суму `prompt_tokens` за прогін із формулою (пункти (а)–(в) кроку 4). Чи збігається зростання з очікуваним?

Для прогонів із рівномірним кроковим приростом збіг повний: у `probe_none_r1` (`C = 1 206`, `d = 748`, `n = 2`) фактична сума `3 160` дорівнює формулі `2*1206 + 748 = 3 160`; у `main_none_r1` (`3 153` == `3 153`). У 3-кроковому `main_none_r3` (`1 206 -> 2 093 (+887) -> 2 220 (+127)`, медіана `d = 507`) формула дає `5 139` проти фактичних `5 519` (різниця 6,9%) через різний розмір відповідей на кожному кроці.

3. Що втрачає агент при обрізанні? Наведіть приклад повторного виклику після стиснення. Що змінилося в `trim+state` (таблиці 3 і 5)?

При `trim` агент втрачає попередні виклики й відповіді інструментів. У `logs/main_trim_r1.log` після `СТИСНЕННЯ: 15 -> 2` на кроці 2 модель повторно викликала `search_books({"query":"Основи алгоритмів"})` та `book_info({"title":"Основи алгоритмів"})` і загалом зробила `288` повторів за 25 кроків (`30 150` вхідних токенів, `step_limit`). У `trim+state` журнал `TaskState` зберіг 12 фактів у `user`-повідомленні: кроки скоротилися з **25 до 6**, повтори — з **288 до 12**, вхідні токени — з **30 150 до 12 681** (-58%), а час — з **53.1 с до 14.3 с**.

4. Яка конфігурація найдешевша за токенами, найшвидша, найточніша (таблиця 5)? Чому висновок за трьома повторами однієї задачі обережний?

За Таблицею 5: **найдешевша** — none (3 160 вхід / 834 вихід), серед методів стиснення — trim+state (12 681 / 2 066, без +6 075 як у summary); **найшвидша** — none (5.9 с), серед стиснень — trim+state (14.3 с); **найточніша** — none (1 з 3 правильно проти 0 з 3 у решти). Висновок обережний через малу вибірку (n = 3), викиди нескінченної генерації (625.2 с у main_trim+state_r1 і 631.7 с у extra_summary_r2) та те, що на одній задачі модель використовувала пакетні виклики.

5. Чому під час вимірювання вартості кеш вимкнено? Що станеться з кешем, якщо інструмент поверне інші дані або ви замініте модель під тією самою назвою?

Кеш вимкнено в measure.py, бо інакше серія повторів вимірювала б швидкість диска (~0.7 с) і тиражувала б один результат замість реального інференсу. Якщо інструмент поверне інші дані між кроками, messages зміниться і кеш промахнеться на наступному кроці, але якщо зміниться зовнішня база books.csv між різними запусками або оновиться модель у Ollama під тим самим іменем qwen25-3k, ключ sha256 залишиться старим і кеш поверне **застарілу відповідь**.

6. Які поламки довгого прогону скорочення історії не лікує в принципі?

Скорочення історії не усуває: (1) арифметичні галюцинації без звернення до calculate (158, -2.0, 27.0); (2) нескінченне зациклення генерації всередині одного кроку (~92 900 вихідних токенів до retry_limit); (3) втрату чисел при переказі малою 3B-моделлю в summary (3.5 -> 114); (4) повторні помилки, оскільки TaskState зберігає лише ok-виклики, а невдалі спроби витираються.

9. Відповіді на питання для захисту

1. Чому вартість усього прогону агента зростає швидше, ніж вартість одного кроку? Покажіть це на цифрах свого прогону.

На кожному кроці передається вся попередня історія (C, C+d, C+2d, ...), тому сума зростає квадратично: n*C + d*n*(n-1)/2 (O(n^2)). У probe_none_r1: крок 1 = 1 206, крок 2 = 1 954 (+748), сума = 3 160 токенів. У main_trim+state_r3 до стиснення: 1 206 -> 1 762 -> 2 479 -> 2 725 -> 2 971, сума за 6 кроків сягнула 12 681 токенів.

2. Що відбувається, коли історія не вміщується у вікно? Як ви помітили це у своєму лозі і чому агент не отримує помилки?

Ollama мовчки витісняє найстаріші токени з початку вікна (`system` і задачу), тому помилка не повертається, `prompt_tokens` перестає рости біля межі `NUM_CTX`, але агент забуває ціль і зациклюється.

3. Чому `COMPRESS_AT` менший за `NUM_CTX`?

Тому що у вікні `NUM_CTX = 3072` мають уміститися поточний запит, нові повідомлення кроку та згенерована відповідь `completion_tokens`. Поріг `COMPRESS_AT = 1100` (< `1206`) гарантує запуск стиснення вже після першого пакету викликів.

4. Навіщо цикл `while` у `trim`? Що станеться, якщо його прибрати?

Цикл `while cut < len(rest) and rest[cut]["role"] == "tool"` запобігає появі «осиротілих» відповідей `tool` без попереднього `assistant` із `tool_calls`. Без нього API чат-доповнень відхилить невалідну історію повідомлень.

5. Що втрачає агент при обрізанні і як це компенсує стан? Чого стан не компенсує? Чому стан записується тільки для викликів з рівнем `ok`?

Агент втрачає старі повідомлення інструментів і міркувань (`trim`: 288 повторів). `TaskState` компенсує це компактним списком 12 виконаних викликів (~`544` токени приросту на кроці 2 замість `748`, повтори падають до 11–12), але не зберігає логіку та контекст помилок. Запис лише рівня `ok` не дає хибним викликам (`syntax`/`schema`/`meaning`) засмічувати пам'ять.

6. З чого складається ключ кешу? Коли від кешу є користь, а коли його не можна вмикати?

Ключ `sha256` обчислюється від `{model, messages, tools, temperature}`. Корисний при налагодженні та регресійних прогонах (`12.6×` прискорення: `0.7 с` проти `8.8 с`); заборонений при вимірюванні реальної вартості (`measure.py`), зміні БД чи заміні моделі під старим ім'ям.

7. Чому кожну конфігурацію проганяли тричі й брали медіану? Чому висновки все одно обережні?

Медіана згладжує одиничні збої на кшталт `625.2 с` (`main_trim+state_r1`) або `631.7 с` (`extra_summary_r2`), але 3 повтори на одній задачі дають обмежену статистичну потужність за умов пакетної стратегії 3B-моделі.

8. Покажіть у трасі Langfuse місце, де стиснення зменшило вхід моделі.

У трасі `main_trim+state_r3` (`8faa9e61b8520bbdfe63edbfd96fa360`) між 5-м і 6-м вузлами `OpenAI-generation`: вхід на кроці 5 сягнув `2 971` токен (97% від `3 072`), після

стиснення `11 -> 8` повідомлень із `TaskState` на кроці 6 він зменшився до **`1 538` токенів** (`-1 433`).

10. Висновок

У ході лабораторної роботи №4 для агента бібліотечної системи (варіант 11, модель `qwen25-3k`, `NUM_CTX = 3072`, `COMPRESS_AT = 1100`, логування через `utils/lab_logger.py`) підтверджено, що кешування прискорює повторні прогони у **12,6 раза** (`0.7 с` проти `8.8 с`), чисте обрізання `trim` зациклюється через амнезію (`25` кроків, `288` повторів), а поєднання обрізання зі станом `trim+state` скорочує кроки у **4,2 раза** (`6` проти `25`), повтори — у **24 рази** (`12` проти `288`) та вхідні токени — на **58%** (`12 681` проти `30 150`). Семантичне підсумовування `summary` на 3B-моделі виявилося дорожчим (`15 420 + 6 075` токенів у `main`, `12 089 + 2 895` у `extra`) і менш точним за детермінований журнал `TaskState` через втрату та спотворення чисел під час переказу.